



**Instituto Tecnológico Superior
de san Pedro**

Análisis

Docente: Ruth Aivi Chávez Rodríguez

“Ingeniería en sistemas computacionales”

Jesús Antonio Montoya Alvarado

5“A”

Análisis del caso C01: Sistema de citas de una clínica

1. Presentación del caso y consecuencias

La clínica utiliza un sistema web para registrar citas médicas mediante la selección del paciente, médico, fecha y horario. Aunque el sistema permite guardar una cita correctamente, presenta varios problemas que demuestran que un software no debe evaluarse solamente porque su pantalla funcione. Dos recepcionistas pueden reservar el mismo horario casi al mismo tiempo, algunos cambios se comunican por mensajería sin actualizar el sistema y tampoco existe un registro claro de quién modifica o cancela una cita.

Además, el equipo desarrollador publica cambios directamente en producción cuando surge un problema urgente. Las pruebas realizadas son muy limitadas, ya que solamente comprueban que la pantalla abra y que sea posible guardar una cita. Tampoco existe un procedimiento adecuado para respaldar información, recuperar una versión anterior o capacitar al personal cuando cambia el flujo de trabajo.

Las consecuencias afectan a diferentes personas. Para los pacientes pueden presentarse citas duplicadas, confusión y cambios de horario. Para los recepcionistas y médicos significa más trabajo, discusiones y tiempo dedicado a corregir información. Para los desarrolladores existe un mayor riesgo de introducir nuevos errores al modificar directamente el sistema. Para la clínica, estos problemas disminuyen la confiabilidad de la información y afectan la organización del servicio.

2. Cinco problemas identificados

Problema 1: Citas duplicadas

Causa: El sistema no controla correctamente que dos usuarios intenten reservar el mismo horario simultáneamente.

Consecuencia: Dos pacientes pueden quedar registrados para un mismo horario, provocando conflictos entre recepción, médicos y pacientes.

Práctica de Ingeniería de Software: Validación de reglas de negocio, integridad de datos y pruebas de concurrencia.

Evidencia del caso: Se menciona que dos recepcionistas pueden reservar el mismo horario casi al mismo tiempo y que han aumentado las citas duplicadas.

Problema 2: Información desactualizada

Causa: Los cambios pueden comunicarse mediante mensajería y no registrarse posteriormente en el sistema.

Consecuencia: Diferentes personas pueden trabajar con información distinta y se requiere corregir manualmente los datos.

Práctica: Centralización de la información y establecimiento de un procedimiento para actualizar las citas.

Evidencia: El caso indica que el personal puede avisar por mensajería y no siempre actualiza el sistema.

Problema 3: Falta de trazabilidad

Causa: No existe un registro de quién modificó o canceló una cita.

Consecuencia: Cuando aparece un problema, resulta difícil conocer qué ocurrió y cuándo se realizó el cambio.

Práctica: Implementar una bitácora de auditoría que registre usuario, fecha, hora y acción realizada.

Evidencia: El caso señala que no existe un registro claro de quién modificó o canceló una cita.

Problema 4: Cambios directos en producción

Causa: Los desarrolladores publican directamente los cambios cuando la clínica reporta problemas urgentes.

Consecuencia: Una modificación que busca solucionar un error puede generar otros problemas en el sistema.

Práctica: Utilizar control de versiones y separar los ambientes de desarrollo, pruebas y producción.

Evidencia: El caso menciona que el equipo publica cambios directamente en producción.

Problema 5: Pruebas y mantenimiento insuficientes

Causa: Las pruebas solamente comprueban que la pantalla abra y permita guardar una cita. También faltan procedimientos de respaldo, recuperación y capacitación.

Consecuencia: Los errores pueden llegar a producción y los empleados pueden tener dificultades para utilizar nuevos flujos de trabajo.

Práctica: Realizar pruebas funcionales, de regresión y de concurrencia, además de implementar respaldos y capacitación.

Evidencia: El caso indica que las pruebas son limitadas y que no existe un procedimiento adecuado de respaldo, recuperación o capacitación.

3. Tres acciones prioritarias

Primera acción: controlar las citas duplicadas

Esta debe ser la primera acción porque afecta directamente a pacientes y personal. El sistema debe impedir que dos usuarios confirmen simultáneamente el mismo médico, fecha y horario.

El costo sería modificar parte del sistema y realizar pruebas con varios usuarios. La mejora se comprobaría intentando reservar el mismo horario desde dos cuentas diferentes. El sistema debería aceptar solamente una reserva y rechazar la segunda.

Segunda acción: establecer pruebas y control de cambios

La segunda acción consiste en dejar de publicar modificaciones directamente en producción. Los cambios deben probarse primero en un ambiente separado. Esto reduce la posibilidad de que una solución genere nuevos problemas.

La principal limitación es que se necesitará tiempo adicional para realizar las pruebas y mantener un ambiente de prueba. Para comprobar la mejora, se puede registrar cada cambio y verificar que haya sido probado antes de llegar al sistema utilizado por la clínica.

Tercera acción: implementar auditoría, respaldos y capacitación

La tercera acción consiste en registrar las modificaciones y cancelaciones, realizar respaldos periódicos y capacitar al personal cuando cambie el funcionamiento del sistema.

El costo incluye almacenamiento, configuración y tiempo para capacitar a los empleados. La mejora se comprobaría revisando las bitácoras, realizando pruebas de restauración y observando si el personal utiliza correctamente los nuevos procedimientos.

4. Reflexión personal

Este caso me permite comprender que desarrollar software no significa únicamente hacer que un programa funcione. Un sistema puede abrir correctamente y guardar información, pero todavía presentar problemas importantes si no se consideran situaciones reales de trabajo.

Si yo desarrollara un sistema parecido, realizaría pruebas más completas y no solamente comprobaría que cada pantalla funciona. También probaría situaciones como dos usuarios intentando reservar el mismo horario, modificaciones simultáneas y recuperación de información después de un error.

Además, utilizaría un proceso más controlado para publicar cambios y mantendría registros de las acciones importantes. También considero importante escuchar a los usuarios, porque ellos conocen problemas que pueden no aparecer durante las pruebas realizadas por los desarrolladores.

En conclusión, el caso demuestra que la calidad del software depende de mucho más que su funcionamiento básico. Es necesario considerar la información, los usuarios, los procesos, las pruebas, el mantenimiento y la seguridad. Un buen desarrollo debe buscar prevenir problemas antes de que lleguen a afectar a las personas que utilizan el sistema.

Fuentes consultadas

- ISO – International Organization for Standardization. (2023). ISO/IEC 25010:2023 — Systems and software engineering — Product quality model. Esta norma aborda la calidad de los productos de software y su evaluación durante el ciclo de vida.
ISO/IEC 25010:2023
- NIST – National Institute of Standards and Technology. (s. f.). Software Testing. Información sobre pruebas de software y métodos para evaluar sistemas.
NIST – Software Testing
- Black, P. E., Okun, V., & Guttman, B. (2021). Guidelines on Minimum Standards for Developer Verification of Software (NISTIR 8397). NIST.

Incluye recomendaciones sobre técnicas de verificación y pruebas de software.

NISTIR 8397

- Wallace, D., Ippolito, L. M., & Cuthill, B. B. (1996). Reference Information for the Software Verification and Validation Process (NIST SP 500-234). NIST. NIST SP 500-234

Declaración sobre el uso de inteligencia artificial

Para elaborar este análisis se utilizó inteligencia artificial como apoyo para organizar y redactar las ideas a partir de la información proporcionada en las imágenes. Las conclusiones se basan en el caso presentado y las fuentes consultadas se utilizaron para complementar los conceptos relacionados con calidad y pruebas de software.